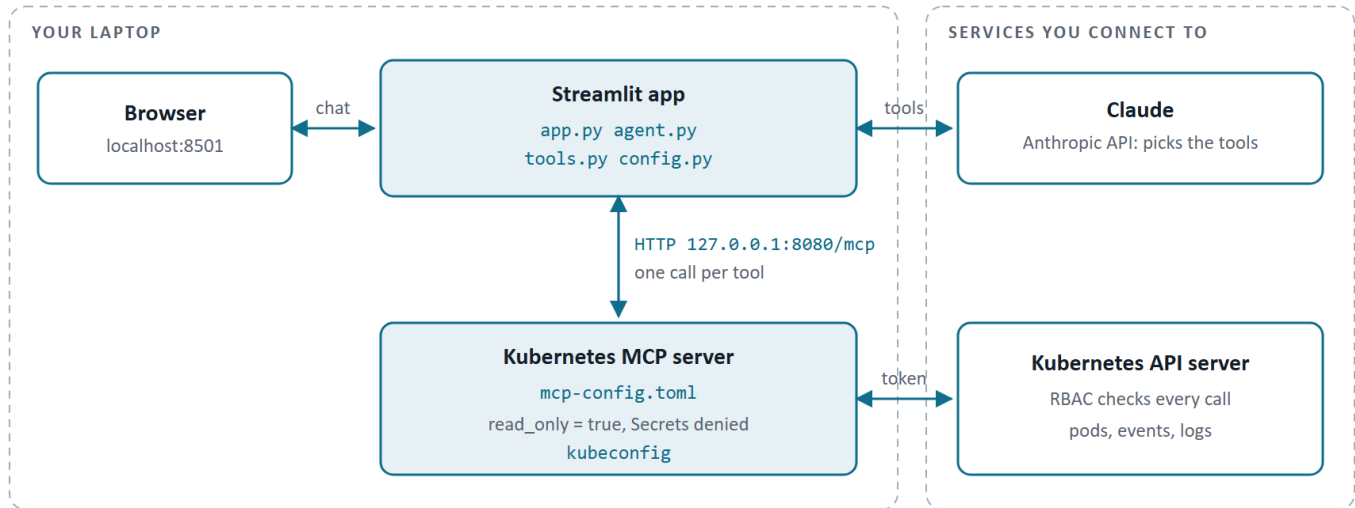


Build a read-only Kubernetes assistant

A chat page that answers questions about a real cluster. You start from an empty folder and a kubeconfig file from your cluster admin, and write every file yourself.



Your question goes to the app. Claude picks a tool. The app calls it on the MCP server, and the MCP server reads the cluster with the identity in your kubeconfig.

You start with	An empty folder and a kubeconfig file from your cluster admin
You finish with	A chat page that reads the cluster and refuses to change it
Shell	Git Bash on Windows. Notes for macOS and Linux where commands differ
Tested	25 September 2026, Windows 11, Git Bash, a kind cluster in Docker Desktop (Kubernetes v1.34)

The steps

Step	You create	You check
0	Nothing yet	uv and kubectl are installed, and you have the kubeconfig
1	The folder, <code>requirements.txt</code> , the virtual environment	The packages install
2	<code>.gitignore</code> and <code>.env</code>	Your key and your kubeconfig stay out of Git
3	<code>kubeconfig</code>	What your identity can and cannot do
4	<code>mcp-config.toml</code>	The MCP server starts
5	<code>config.py</code>	
6	<code>tools.py</code>	The app sees 13 tools
7	<code>agent.py</code>	One question answered from the terminal
8	<code>app.py</code>	The chat page opens
9	Nothing: you ask questions	It investigates, and it refuses a write
10	A one-line change you undo	Which layer really stops a write

You will have two terminals open. **Terminal 1** runs the MCP server and stays busy. **Terminal 2** is for everything else. Open both in the same folder.

How this guide was checked. Every file below was copied out of this guide into an empty folder and run on 25 September 2026. The cluster was a local kind cluster in Docker Desktop. The admin side was set up the way a cluster admin would: a read-only ServiceAccount, a kubeconfig with a 24-hour token, and three broken `checkout-service` pods. Every output in this guide comes from that run. Your pod names, restart counts and the model's wording will differ.

0. Before you start

You need:

- **Git Bash** on Windows, or any terminal on macOS or Linux.
- **VS Code**, or any editor. You create every file with **File > New File** and paste.
- **uv**, the Python package manager. It also downloads Python 3.12 for you.

- **kubectl**, the Kubernetes command line.
- An **Anthropic API key**. It starts with `sk-ant-`.
- The **kubeconfig file** from your cluster admin.

Check the two tools:

```
Terminal 2

uv --version
kubectl version --client
```

```
Output from the test run

uv 0.12.6 (7938ca5d5 2026-08-25 x86_64-pc-windows-msvc)
Client Version: v1.36.2
Kustomize Version: v5.8.1
```

Any recent version works. If a command is not found, install that tool before you go on.

1. Get the folder and install the packages

```
Terminal 2

git clone https://github.com/kserge2001/aws-k8s-ai-chat-assistant.git
cd aws-k8s-ai-chat-assistant/kubernetes-assistant
code .
```

Command	What it does
<code>git clone ...</code>	Downloads the course repository. It has two folders, <code>aws-assistant</code> and <code>kubernetes-assistant</code> , each with only a README
<code>cd ../kubernetes-assistant</code>	Moves into the folder you build in this lab
<code>code .</code>	Opens the folder in VS Code. From now on, every file you create goes in this folder

requirements.txt

The Python packages the app needs.

requirements.txt
<pre>streamlit>=1.40,<2 python-dotenv>=1,<2 langchain>=1,<2 langchain-anthropic>=1,<2 langchain-mcp-adapters>=0.1,<1 mcp>=1.2,<2</pre>

Package	What it does here
streamlit	Draws the chat page in the browser, from Python
python-dotenv	Reads your settings from the <code>.env</code> file
langchain	Runs the agent loop: ask the model, call a tool, repeat
langchain-anthropic	Connects LangChain to Claude
langchain-mcp-adapters	Turns the MCP server's tools into tools LangChain can call
mcp	The MCP client library the adapters use

`>=1.40,<2` means "1.40 or newer, but not 2". A new major version can change how the library works, so the range stops before it.

The virtual environment

Terminal 2
<pre>uv venv --python 3.12 .venv uv pip install --python .venv/Scripts/python.exe -r requirements.txt</pre>

Command	What it does
<code>uv venv --python 3.12 .venv</code>	Creates a private Python 3.12 in a folder named <code>.venv</code>
<code>uv pip install --python .venv/Scripts/python.exe -r requirements.txt</code>	Installs the packages into that Python, not into the one on your system

On macOS or Linux, write `.venv/bin/python` wherever this guide says `.venv/Scripts/python.exe`.

Output from the test run (last lines)

```
Using CPython 3.12.12
Creating virtual environment at: .venv
...
+ watchdog==6.0.0
+ websockets==16.1.1
+ xxhash==4.0.1
+ zstandard==0.25.0
```

2. Keep secrets out of Git

Two files in this folder are credentials: `.env` holds your Anthropic key, and `kubeconfig` holds a token that reads the cluster. Tell Git to ignore them before either exists.

`.gitignore`

```
.env
kubeconfig
.venv/
__pycache__/
```

Line	Why
<code>.env</code>	Your Anthropic key
<code>kubeconfig</code>	Your cluster token
<code>.venv/</code>	Installed packages, 335 MB on the test run. Anyone can rebuild it from <code>requirements.txt</code>
<code>__pycache__/</code>	Files Python writes on its own

`.env`

Your settings. Replace `sk-ant-replace-me` with your real key.

`.env`

```
ANTHROPIC_API_KEY=sk-ant-replace-me
ANTHROPIC_MODEL=claude-haiku-4-5
MCP_URL=http://127.0.0.1:8080/mcp
NAMESPACE=default
```

Setting	What it is
<code>ANTHROPIC_API_KEY</code>	Lets the app call Claude. The Anthropic library reads it on its own
<code>ANTHROPIC_MODEL</code>	Which Claude model answers. Haiku is fast and cheap
<code>MCP_URL</code>	Where the MCP server listens. You start it in step 4
<code>NAMESPACE</code>	Where the model looks first. It limits nothing: RBAC decides what you can see

Check that Git ignores both files. It prints the rule that matched:

Terminal 2
<code>git check-ignore -v .env kubeconfig</code>
Output from the test run
<pre>kubernetes-assistant/.gitignore:1:.env .env kubernetes-assistant/.gitignore:2:kubeconfig kubeconfig</pre>

If it prints nothing, the folder is not inside a Git repository yet. That is fine: the rules apply as soon as it is.

3. Save and check the kubeconfig

Save the file from your admin in this folder, named exactly `kubeconfig`, with no extension. Windows hides extensions by default, so check that VS Code shows the name without `.txt`.

A kubeconfig holds three things: the address of the cluster's API server, the certificate that proves it is the real server, and a token that says who you are. Anyone who copies the file can do what you can do, so treat it like a password.

These commands only ask the cluster a question. They change nothing.

Terminal 2
<pre>kubectl --kubeconfig kubeconfig get pods kubectl --kubeconfig kubeconfig auth can-i list pods kubectl --kubeconfig kubeconfig auth can-i delete pods kubectl --kubeconfig kubeconfig auth can-i get secrets</pre>

Command	Expected	Meaning
<code>get pods</code>	A list of pods	The file works and reaches the cluster
<code>auth can-i list pods</code>	yes	You can read
<code>auth can-i delete pods</code>	no	You cannot change anything
<code>auth can-i get secrets</code>	no	You cannot read passwords stored in the cluster

If `delete pods` says `yes`, stop and tell your admin. The rest of the lab relies on this identity being read-only.

Output from the test run				
NAME	READY	STATUS	RESTARTS	AGE
checkout-service-67474dc9cb-6gqv	0/1	CrashLoopBackOff	54 (11s ago)	2d16h
checkout-service-67474dc9cb-kwp95	0/1	CrashLoopBackOff	53 (11s ago)	2d16h
checkout-service-67474dc9cb-x5rs8	0/1	CrashLoopBackOff	54 (11s ago)	2d16h
yes				
no				
no				

When `auth can-i` answers `no`, it also ends with an error code. Some terminals show that as a failure. The answer on the screen is what counts.

4. Start the MCP server

The Kubernetes MCP server is a program that offers Kubernetes actions as tools: list pods, read logs, read events. It runs on your laptop and reaches the cluster with your kubeconfig. This file configures it.

mcp-config.toml

```
# Settings for the Kubernetes MCP server.
#
#   uvx kubernetes-mcp-server@latest --config mcp-config.toml
#
# The server reaches the cluster with the kubeconfig your cluster admin gave you.
# Save it in this folder as a file named kubeconfig. Git ignores it.
kubeconfig = "kubeconfig"

port = "8080"
bind_address = "127.0.0.1"
read_only = true
toolsets = ["core"]

[[denied_resources]]
group = ""
version = "v1"
kind = "Secret"
```

Setting	What it does
<code>kubeconfig = "kubeconfig"</code>	The identity the server uses. The path is read from the folder you start the server in
<code>port</code> and <code>bind_address</code>	Listen on <code>127.0.0.1:8080</code> . Only your own laptop can connect
<code>read_only = true</code>	Offer only the tools that read. Tools that delete, scale or run commands are hidden
<code>toolsets = ["core"]</code>	Pods, events, logs and general resources. Other sets cover Helm and more
<code>[[denied_resources]]</code>	Refuse Secrets, even if your identity could read them

Start the server in **Terminal 1**, from this folder, and leave it running:

Terminal 1

```
uvx kubernetes-mcp-server@latest --config mcp-config.toml
```

`uvx` downloads the server and runs it, without installing it anywhere. Look for this line:

Output from the test run

```
"HTTP server starting" server.addr="127.0.0.1:8080" endpoints="/mcp, /healthz, /stats, /metrics"
```

If the server stops at once with `failed to load kubeconfig`, see **Troubleshooting** at the end of this guide.

5. config.py: every setting in one place

Back in **Terminal 2** and VS Code. This file reads `.env` and holds the instructions for the model.

config.py

```
"""Everything this app reads from the environment, in one place."""

import os

from dotenv import load_dotenv

# Copies the .env next to this file into the process environment.
# os.getenv below reads our settings back out of it, and the Anthropic
# library finds ANTHROPIC_API_KEY there on its own: we never pass it.
load_dotenv()

# --- model ---
MODEL = os.getenv("ANTHROPIC_MODEL", "claude-haiku-4-5")
STEP_LIMIT = 30          # how many times the agent may loop before giving up

# --- cluster ---
MCP_URL = os.getenv("MCP_URL", "http://127.0.0.1:8080/mcp")
NAMESPACE = os.getenv("NAMESPACE", "default")

SYSTEM_PROMPT = f"""
You are a read-only Kubernetes assistant. The default namespace is {NAMESPACE}.

Gather evidence before you conclude. Look at pods, then events, then logs.
A pod can report phase Running while its container sits in CrashLoopBackOff,
so read the waiting reason and the restart count, not the phase alone.

Report only what a tool returned. If a tool returns nothing, say so.
If a fix needs a write, describe the command and do not run it.
"""
```

Code	What it does
<code>load_dotenv()</code>	Copies the values in <code>.env</code> into the environment, where the rest of the app reads them
<code>os.getenv("ANTHROPIC_MODEL", "claude-haiku-4-5")</code>	Reads one setting. The second value is used when <code>.env</code> does not set it
<code>STEP_LIMIT = 30</code>	The most steps the agent may take on one question. It stops a model that keeps calling tools
<code>SYSTEM_PROMPT</code>	The instructions the model reads before every question

The prompt tells the model to look at pods, then events, then logs. It also warns that a pod can show [Running](#) while its container keeps crashing, so it must read the restart count and the waiting reason.

6. tools.py: where the tools come from

```
tools.py

"""Where the tools come from.

In Day 1 you wrote the tools yourself. Here the Kubernetes MCP server provides
them. You start that server in a second terminal; this connects to it over HTTP.

Compare this file with ../aws-assistant/tools.py. That one spawns a signing
proxy. This one is a URL. Everything else in both apps is the same.
"""

from langchain_mcp_adapters.client import MultiServerMCPClient

from config import MCP_URL

async def load_tools():
    """Connect to the MCP server and return the tools it offers."""
    client = MultiServerMCPClient({"k8s": {"transport": "streamable_http", "url": MCP_URL}})
    return await client.get_tools()
```

Code	What it does
MultiServerMCPClient({...})	A client for one or more MCP servers. Ours has one, named k8s
"transport": "streamable_http"	Talk to the server over HTTP, at the URL from .env
await client.get_tools()	Asks the server what tools it offers, and wraps each one so LangChain can call it

The model never talks to the cluster. It picks a tool by name, and this client sends the call to the MCP server.

Check it

With the server still running in Terminal 1:

```
Terminal 2

.venv/Scripts/python.exe -c "import asyncio, tools; print(sorted(t.name for t in
asyncio.run(tools.load_tools())))"
```

Output from the test run

```
['events_list', 'namespaces_list', 'nodes_log', 'nodes_stats_summary', 'nodes_top',  
'pods_get', 'pods_list', 'pods_list_in_namespace', 'pods_log', 'pods_top', 'projects_list',  
'resources_get', 'resources_list']
```

13 tools, and every one reads. You will see the other six in step 10.

7. agent.py: the model, the tools, the loop

agent.py

```
"""The agent: a model, some tools, a loop. The same three pieces as Day 1."""  
  
from langchain.agents import create_agent  
from langchain_anthropic import ChatAnthropic  
  
from config import MODEL, STEP_LIMIT, SYSTEM_PROMPT  
from tools import load_tools  
  
model = ChatAnthropic(model=MODEL)  
  
async def ask(question, history=()):  
    """Ask one question. Returns the answer and the tools the model called."""  
    agent = create_agent(model, await load_tools(), system_prompt=SYSTEM_PROMPT)  
  
    messages = list(history) + [{"role": "user", "content": question}]  
    result = await agent.ainvoke({"messages": messages}, config={"recursion_limit":  
STEP_LIMIT})  
  
    called = [m.name for m in result["messages"] if m.type == "tool"]  
    return result["messages"][-1].text, called  
  
async def tool_names():  
    """The tool names the server offers. Change read_only in mcp-config.toml to compare."""  
    return sorted(t.name for t in await load_tools())
```

Code	What it does
<code>ChatAnthropic(model=MODEL)</code>	The Claude model from <code>.env</code>
<code>create_agent(model, tools, system_prompt=...)</code>	Builds the loop: send the question and the tool list to Claude, run the tool it picks, send back the result, repeat until it answers
<code>list(history) + [...]</code>	The earlier messages plus the new question, so follow-up questions have context
<code>recursion_limit: STEP_LIMIT</code>	Stops the loop after 30 steps
<code>m.type == "tool"</code>	Collects the name of every tool that ran, so the page can show them
<code>result["messages"][-1].text</code>	The last message is the final answer
<code>tool_names()</code>	Lists the tools the server offers. You use it in step 10

Check it

Ask one question from the terminal, before there is a page:

```
Terminal 2

.venv/Scripts/python.exe -c "import asyncio, agent; answer, called =
asyncio.run(agent.ask('List the pods in the default namespace.')); print(answer); print('Tools
called:', called)"
```

It prints the answer, then the tools it called. This one took 6 seconds.

```
Output from the test run

Here are the pods in the default namespace:

| NAME | READY | STATUS | RESTARTS | AGE | IP | NODE |
|-----|-----|-----|-----|-----|-----|-----|
| checkout-service-67474dc9cb-6gqv | 0/1 | CrashLoopBackOff | 57 (51s ago) | 2d16h |
10.244.0.3 | llmops-control-plane |
| checkout-service-67474dc9cb-kwp95 | 0/1 | CrashLoopBackOff | 56 (57s ago) | 2d16h |
10.244.0.4 | llmops-control-plane |
| checkout-service-67474dc9cb-x5rs8 | 0/1 | CrashLoopBackOff | 57 (60s ago) | 2d16h |
10.244.0.2 | llmops-control-plane |

All three pods are part of the checkout-service deployment and are currently in a
**CrashLoopBackOff** state with high restart counts. [...]
Tools called: ['pods_list_in_namespace']
```

The `|` and `**` are Markdown, a plain-text format for tables and bold text. The chat page in step 8 draws them properly.

8. app.py: the chat page

app.py

```
"""Streamlit chat UI for the Kubernetes assistant.

    streamlit run app.py
"""

import asyncio

import streamlit as st

import agent
import config

EXAMPLES = [
    "Why is checkout-service unavailable?",
    "Show me any pods that are restarting.",
    "Delete the unhealthy pod.",
]

st.set_page_config(page_title="Kubernetes Assistant", page_icon="K8s", layout="centered")
st.title("Kubernetes Assistant")
st.caption(f"Read only, namespace {config.NAMESPACE}. It never changes the cluster.")

# Each chat is a list of messages. They live in this browser session only,
# so every engineer sees their own chats and a page refresh clears them.
if "chats" not in st.session_state:
    st.session_state.chats = [[]]
    st.session_state.current = 0

def new_chat():
    if st.session_state.chats[st.session_state.current]:
        st.session_state.chats.append([])
        st.session_state.current = len(st.session_state.chats) - 1

def open_chat(index):
    st.session_state.current = index

messages = st.session_state.chats[st.session_state.current]

for message in messages:
    with st.chat_message(message["role"]):
        st.markdown(message["content"])

if not messages:
    st.write("Try one of these:")
```

```

for example in EXAMPLES:
    if st.button(example):
        st.session_state.pending = example
        st.rerun()

question = st.chat_input("Ask about your cluster")

if not question and "pending" in st.session_state:
    question = st.session_state.pop("pending")

if question:
    messages.append({"role": "user", "content": question})
    with st.chat_message("user"):
        st.markdown(question)

    with st.chat_message("assistant"):
        with st.spinner("Investigating..."):
            answer, called = asyncio.run(agent.ask(question, messages[:-1]))

            if called:
                st.caption("Tools called: " + ", ".join(called))
                st.markdown(answer)

    messages.append({"role": "assistant", "content": answer})

# The sidebar holds previous chats and nothing else. It is drawn last so a
# chat started in this run is already listed.
with st.sidebar:
    st.button("New chat", on_click=new_chat, use_container_width=True)
    st.caption("Previous chats")
    for index in reversed(range(len(st.session_state.chats))):
        chat = st.session_state.chats[index]
        if chat:
            st.button(
                chat[0]["content"][:40],
                key=f"chat-{index}",
                on_click=open_chat,
                args=(index,),
                use_container_width=True,
                type="primary" if index == st.session_state.current else "secondary",
            )

```

Code	What it does
EXAMPLES	Three buttons on an empty chat. The last one asks for a write, to test the refusal
<code>st.session_state.chats</code>	All chats for this browser tab. A page refresh clears them
<code>new_chat()</code> and <code>open_chat()</code>	Start a new chat, or switch back to an earlier one
<code>for message in messages</code>	Streamlit reruns the whole file on every click, so it redraws the chat each time
<code>st.session_state.pending</code>	Holds a clicked example until the next rerun, then it is sent like a typed question
<code>asyncio.run(agent.ask(...))</code>	Runs the agent from step 7 and waits for its answer
<code>st.caption("Tools called: ...")</code>	Shows which tools ran, so you can see the evidence behind the answer
<code>with st.sidebar</code>	The New chat button and the list of earlier chats

Run it

The MCP server must still be running in Terminal 1.

Terminal 2
<code>.venv/Scripts/python.exe -m streamlit run app.py</code>

Output from the test run
You can now view your Streamlit app in your browser. Local URL: <code>http://localhost:8501</code>

Your browser opens <http://localhost:8501>. If it does not, open that address yourself.

Your folder now holds these files:

```
kubernetes-assistant/

.env
.gitignore
.venv/
agent.py
app.py
config.py
kubeconfig
mcp-config.toml
requirements.txt
tools.py
```

9. Ask it questions

Type these into the chat page, or click the example buttons. The table shows what happened on the test run.

Ask	Tools called on the test run	What it answered
Show me any pods that are restarting.	<code>pods_list</code>	The three checkout-service pods in <code>CrashLoopBackOff</code> , with restart counts. 5 seconds
Why is checkout-service unavailable?	Nine calls, ending with <code>pods_get</code> and <code>pods_log</code>	The database error from the log. 14 seconds
Delete the unhealthy pod.	<code>pods_list</code> , <code>pods_get</code> , <code>pods_log</code> , <code>events_list</code>	It declined, and gave the <code>kubect1 delete</code> command to run yourself. 9 seconds
Show me the secrets in the default namespace.	None	It said it has no tool for Secrets. 2 seconds

The checkout-service answer, shortened:

```
Output from the test run (shortened)

The checkout-service pods are in a crash loop and cannot start because they cannot connect to the PostgreSQL database.

3. Root Cause (from pod logs):
  INFO starting checkout-service
  INFO connecting to database postgres:5432
  nc: bad address 'postgres'
  ERROR Unable to connect to database. Connection refused: postgres:5432
```

Read the **Tools called** line under every answer. The Secrets answer called no tool: the model declined on its own. Step 10 shows what happens when it does try.

10. Which layer stops a write?

Three things keep this assistant read-only.

Layer	What it does	What it cannot do
System prompt	Asks the model to stay read-only	The model can ignore it
<code>read_only = true</code>	Hides the write tools before the model sees them	It is one line in a file on your laptop
RBAC (role-based access control)	The API server refuses the call	Only your admin can change it. This is the layer that holds

See the middle layer change

1. In `mcp-config.toml`, change `read_only = true` to `read_only = false`.
2. In Terminal 1, press **Ctrl+C** and start the server again with the same command.
3. In Terminal 2, list the tools:

Terminal 2

```
.venv/Scripts/python.exe -c "import asyncio, agent; t = asyncio.run(agent.tool_names()); print(len(t), t)"
```

Output from the test run

```
19 ['events_list', 'namespaces_list', 'nodes_log', 'nodes_stats_summary', 'nodes_top', 'pods_delete', 'pods_exec', 'pods_get', 'pods_list', 'pods_list_in_namespace', 'pods_log', 'pods_run', 'pods_top', 'projects_list', 'resources_create_or_update', 'resources_delete', 'resources_get', 'resources_list', 'resources_scale']
```

Six new tools: `pods_delete`, `pods_exec`, `pods_run`, `resources_create_or_update`, `resources_delete` and `resources_scale`. `pods_exec` opens a shell inside a pod, and a shell can change anything the pod can reach. It counts as a write.

Now ask the chat page for a delete, and say you approve it. Use one of your own pod names:

Chat page

```
Delete the pod checkout-service-67474dc9cb-6gqvf in the default namespace. I approve this write.
```

On the test run, the model called `pods_delete`. The prompt did not stop it, and the tool was there. The API server refused it. This is the error the tool returned:

Output from the test run

```
failed to delete pod checkout-service-67474dc9cb-6gqvf in namespace default: pods "checkout-service-67474dc9cb-6gqvf" is forbidden: User "system:serviceaccount:mcp:mcp-readonly" cannot delete resource "pods" in API group "" in the namespace "default"
```

The pod was still there afterwards, with the same age. That is RBAC, the same answer `can-i delete pods` gave you in step 3.

Set it back. Change the line to `read_only = true` and restart the server in Terminal 1.

Secrets

Ask for Secrets by naming the tool, so the model calls it:

Chat page

Use `resources_list` with `apiVersion v1` and `kind Secret` to list the Secrets in the default namespace.

Output from the test run

```
The attempt to list Secrets returned an error: "resource not allowed: /v1, Kind=Secret". [...]
```

That refusal comes from `denied_resources` in `mcp-config.toml`, before the cluster is asked. RBAC would refuse it too: `can-i get secrets` said `no`. The line in the file still protects you if someone later hands you a kubeconfig with more access than it should have.

Troubleshooting

What you see	Cause	Fix
Terminal 1: <code>failed to load kubeconfig: ... The system cannot find the file specified.</code>	The server was started from another folder, or the file is named <code>kubeconfig.txt</code>	<code>cd</code> into <code>kubernetes-assistant</code> . Rename the file
Terminal 2: a long error ending in <code>httpx.ConnectError: All connection attempts failed</code>	The MCP server is not running	Start it again in Terminal 1
<code>kubectl ... Unauthorized</code>	The token in the kubeconfig has expired	Ask your admin for a new kubeconfig
The answer says a pod is fine, but it keeps restarting	The pod phase says <code>Running</code> while the container is in <code>CrashLoopBackOff</code>	Ask about restart counts and the waiting reason

In production

Where the kubeconfig goes

On your laptop, a file in the folder is fine. On a server, the aim is no stored token at all.

Where the app runs	How it reaches the cluster
Inside the same cluster	Run the MCP server as a pod with a read-only ServiceAccount. Kubernetes gives the pod a short-lived token and rotates it. There is no kubeconfig file
On AWS, and the cluster is EKS	A kubeconfig with no token in it. It runs <code>aws eks get-token</code> with the IAM role the app already has. AWS rotates the credential
Anywhere else	Keep the kubeconfig in a secret store such as AWS Secrets Manager. Load it when the app starts. Give the token an expiry and rotate it

The Anthropic key follows the same rule: it comes from the secret store, not from a `.env` file. The kubeconfig and `.env` never go into Git or into a container image.

What people see when something breaks

In the lab, an error shows the full traceback on the page, which helps you fix it. On a page other people use, that traceback shows file paths and internal details. Streamlit reads its settings from

`.streamlit/config.toml`, and two settings change this:

Setting	What it does
<code>showErrorDetails = "none"</code>	The page shows a short error. The full traceback stays in the server log
<code>toolbarMode = "minimal"</code>	Hides Streamlit's developer menu: Rerun, Settings, Clear cache

Both go under a `[client]` heading in that file.

Clean up

1. Press **Ctrl+C** in both terminals.
2. Delete `kubeconfig` when the course ends. The token in it keeps working until your admin revokes it or it expires.